Farm Route Planning as a Directed Arc-Routing Problem

1 Purpose

This document specifies construction and solution of the farm-level route-planning problem.

The planner takes a farm task containing an ordered list of fields to service, an implement configuration, a start pose, and an end pose. It produces a single tractor route through crop-row service paths, field maneuvering regions, and the farm road network. The listed fields are completed in order.

The optimization is formulated as a directed arc-routing problem and solved using OR-Tools CP-SAT. The purpose of CP-SAT is to determine which directed graph arcs the tractor traverses, and how many times each arc is traversed, such that:

1. every required farming task is serviced at least once;
2. service in each listed field is completed before service in the next field begins;
3. the tractor starts at the specified start pose;
4. the tractor finishes at the specified end pose;
5. the selected traversals form one connected route;
6. all movement is through explicitly permitted graph arcs; and
7. a distance-equivalent route cost is minimized.

The geometric generation of road arcs, crop-row arcs, road-junction turns, and Reeds–Shepp (RS) field connections occurs before CP-SAT is invoked. Each movement arc contains a traversal cost and the geometric tractor path associated with that traversal. The directed road graph is then copied into ordered field-service stages before optimization.

CP-SAT determines how many times each directed graph arc is traversed. The selected arc multiplicities are then converted into an ordered tractor route by constructing the corresponding directed multigraph and computing an Euler trail from the specified start vertex to the specified end vertex. The stored geometries of the ordered arcs are concatenated to produce the farm-level tractor path.

Thus the route-generation pipeline is

1. farm geometry and task
2. directed routing graph
3. CP-SAT traversal optimization
4. directed Euler-trail reconstruction
5. tractor path

The resulting path is subsequently supplied to the vehicle trajectory-following or control subsystem for execution.

2 Farm Task

The input farm task is

$$\mathcal{F} = (\mathcal{L}, \tau, p_s, p_t)\,,$$

where

- $\mathcal{L} = (L_1, \ldots, L_m)$ is the ordered list of distinct fields to be serviced;
- τ is the field task;
- p_s is the tractor start pose;
- p_t is the tractor end pose.

For the current implementation,

$$\tau = (\text{ImplementSide}, \text{ImplementRange})\,,$$

with

$$\text{ImplementSide} \in \{\text{Left}, \text{Right}, \text{Both}\}.$$

The default implement range is

$$\text{ImplementRange} = \frac{\text{row spacing}}{2}.$$

A pose is an SE(2) tractor pose,

$$p = (x, y, \theta).$$

3 Logical Service Requirements

The set of farming operations that must be completed is denoted

$$\mathcal{Q} = \{q_1, q_2, \ldots, q_K\}.$$

An element $q \in \mathcal{Q}$ is a *logical service requirement*. It is not itself a graph node or graph edge. For example, a requirement can mean:

> Inspect this side of this tree row using the left-mounted implement.

or

> Traverse a path from which this tree-row side lies within the range of an implement mounted on both sides.

The geometry generator determines which directed tractor traversals satisfy each requirement. The separation between logical tasks and physical paths is important.

Several service requirements may be satisfied by one physical tractor traversal. Conversely, the same physical path may be traversed in the opposite direction without satisfying a particular service requirement.

3.1 One-sided implements

For a left- or right-mounted implement, direction matters. Suppose a physical row path has endpoints A and B. Both physical traversals may be legal:

$$A \to B$$

and

$$B \to A.$$

However, only one of these directions may place the trees on the implement side of the tractor. Thus both arcs may exist in the travel graph, while only one is marked as satisfying the corresponding service requirement.

This permits, for example,

$$A \to B$$

to perform the farming task, followed later by

$$B \to A$$

as a deadhead traversal needed to leave the field.

3.2 Overlapping tasks

For `ImplementSide=Both`, candidate service paths associated with neighboring tree rows may overlap. Logical service requirements must not be discarded merely because their physical paths overlap. Instead, geometrically equivalent physical paths are canonicalized into a single graph arc, and that graph arc records every service requirement it satisfies.

Thus one traversal can satisfy several requirements simultaneously.

4 Directed Routing Graph

Construct a directed multigraph

$$G = (V, A).$$

Multiple directed arcs between the same pair of graph vertices are permitted.

Each vertex represents a routing state. Each directed arc represents one physically permitted tractor movement. Partition the arc set as

$$A = A_{\text{road}} \cup A_{\text{row}} \cup A_{\text{RS}} \cup A_{\text{turn}} \cup A_{\text{switch}}.$$

Here A_{RS} contains field maneuvers, A_{turn} contains explicit road-junction maneuvers, and A_{switch} contains zero-motion changes of field-service stage. Additional arc types may be introduced later without changing the optimization formulation. Each arc $a \in A$ stores at least

$$a = (u_a, v_a, \gamma_a, \ell_a, T_a, F_a, C_a)\,,$$

where

- $u_a \in V$ is the tail vertex;
- $v_a \in V$ is the head vertex;
- γ_a is the geometric path followed by the tractor;
- ℓ_a is the path length in meters;
- T_a is the arc type;
- F_a is an optional field identifier; and
- $C_a \subseteq \mathcal{Q}$ is the set of service requirements automatically satisfied when this arc is traversed.

For road, RS transit, road-turn, and stage-switch arcs,

$$C_a = \emptyset.$$

For crop-row traversals, C_a may contain one or more service requirements.

5 Graph Vertices and Tractor Pose

A graph vertex is a routing state rather than necessarily a unique (x, y) location.

Where heading matters, especially at row endpoints, two vertices may occupy the same position while having different headings.

For example, the same physical row endpoint can have states

$$(x, y, \theta)$$

and

$$(x, y, \theta + \pi).$$

These must not be merged because they represent different tractor states and have different RS connections.

Every navigable road split has separate states for its two travel headings. Road intersections themselves are not navigable position-only vertices: road arcs end at oriented setback states, and an explicit turn arc connects an incoming leg to an outgoing leg. Field row endpoints and field/road RS interfaces likewise preserve heading.

6 Road Graph Construction

The road network exists independently of the set of fields being serviced.

Split road geometry at

- setback points on each road incident to an intersection;
- field-road connection points;
- StartPose or EndPose projections onto a road; and
- other topology changes.

A bidirectional road section between vertices u and v produces two directed arcs:

$$u \to v$$

and

$$v \to u.$$

Both arcs normally have geometric length equal to the road polyline length; their integer costs follow the formula in the Arc Costs section.

A one-way road creates only the permitted directed arc. Road arcs remain available even when the road passes through a field that is not in the current `field_list`.

No non-road field transit is created in an unselected field.

For an intersection, let d_{setback} be the configured distance from the junction along each incident road. The `PlannerConfig` field is `road_turn_setback_m`; its CLI option is `--road-turn-setback` and its current default is 2.7 m. Replace the direct meeting of road centerlines at the junction by directional road states at those setback points. For each ordered pair of distinct incident legs, connect the incoming state to the outgoing state only if a forward-only RS trajectory is feasible. Thus a three-leg junction has up to six directed turn candidates. A same-leg U-turn is not inserted. If a crossing section is shorter than $2d_{\text{setback}}$, use its midpoint for the setbacks at both ends of that section.

Each road-turn arc stores the sampled rear-axle trajectory and the tractor's swept footprint is checked along it. The allowed region consists of drivable road and crossing surfaces plus the permitted headlands and sidelands of adjacent fields; irrigation channels are excluded except where a GIS crossing surface bridges them. Tree obstacles and the interiors of tree-row convex hulls remain forbidden to the rear-axle trace. An infeasible turn is absent from the graph, so CP-SAT cannot choose it. The original intersection point does not provide a bypass around this check.

Field/road interface candidates are formed by projecting each endpoint of a generated drivable crop lane onto nearby frontage roads, then adding offsets of $\pm d_{\text{interface}}$ along those roads. The `PlannerConfig` field is `road_interface_offset_m`; its CLI option is `--road-interface-offset` and its current default is 2.7 m. It is independent of d_{setback}, despite their equal defaults. These are endpoints of the tractor rear-axle service paths after offsetting and merging tree-row geometry, not endpoints of the tree-row centerlines themselves. Candidates within a junction setback are omitted. Each interface has a state for each road travel heading, allowing a field RS arc to enter or leave a lane without a zero-motion reversal on the road.

6.1 Road anchors for field access

These interface points are deliberately sampled on the existing road centerlines within a selected field; they are not a uniform grid of road samples. For each oriented drivable-lane endpoint, the current builder examines up to two nearby frontage-road sections, projects the endpoint onto each section, and tries the projection plus the two longitudinal offsets. A legal point splits its road arc and acquires separate forward- and backward-heading states. Directed field RS arcs are then tested from each road state to the lane endpoint and from the lane endpoint back to each road state. CP-SAT can therefore choose the road–field–road entry and exit locations along with the row-service route. The offset samples give a short or partial lane room for a forward entry or exit when its direct projection would demand a tight U-turn. They also limit graph size compared with sampling every road at a fixed interval.

One possible refinement is to add road anchors where the extensions of nearby tree-row centerlines intersect the frontage road. For regular rows, these intersections lie roughly half a row spacing from a drivable-lane projection and may provide entry and exit room tailored to the local geometry, analogous to the setback used for junction turns. They would be additional candidate road poses, not tractor paths along the tree rows or mandatory setbacks. The lane projection remains useful for partial rows, merged lanes, and outer lanes where a tree-row extension may be misleading or unavailable. Any such anchors would still need the junction-setback filter and the same RS swept-footprint feasibility check; their effect on route quality and graph size remains to be tested.

7 Crop-Row Traversal Arcs

For every selected field, generate tractor-path centerlines from the tree-row geometry and the implement configuration. Candidate tractor paths are obtained by offsetting the appropriate tree-row centerline by `ImplementRange`. Each canonical physical row path produces directed traversal arcs in each physically legal direction.

For a physical path with endpoint states u and v, these may be

$$a^{+} : u \to v$$

and

$$a^{-} : v' \to u',$$

where u' and v' represent the opposite-heading tractor states at the same physical endpoint locations.

The service-coverage sets

$$C_{a^+} \quad \text{and} \quad C_{a^-}$$

are determined by implement geometry.

For a one-sided implement, one direction may satisfy a service requirement while the reverse direction is merely a legal transit traversal.

For a two-sided implement, one physical traversal may satisfy several service requirements simultaneously.

7.1 Canonicalization of overlapping row paths

Candidate row paths that describe the same physical directed tractor trajectory should be merged into one physical arc.

Path equivalence should use configurable tolerances for

- endpoint position;
- endpoint heading; and
- path geometry.

When equivalent candidate arcs are merged, their coverage sets are unioned:

$$C_a = C_{a_1} \cup C_{a_2} \cup \cdots .$$

Do not merge arbitrary graph vertices merely because their Euclidean positions are close. Vertex merging means the tractor may transition between those routing states with zero movement and is therefore a semantic operation, not merely geometric deduplication.

8 Reeds–Shepp Connection Arcs

For each selected field, form a set of field interface states consisting of appropriate

- row endpoint poses;
- field-road connection poses;
- StartPose, if located in the field; and
- EndPose, if located in the field.

For candidate ordered pairs of interface states (p_i, p_j) in the same selected field:

1. generate RS candidates at the tractor's minimum turning radius and configured larger radii;
2. rank candidates by length, reverse distance, internal gear changes, and reverse episodes;
3. check the swept tractor footprint and rear-axle trace along each candidate;
4. reject candidates that violate field, channel, tree, or tree-hull constraints; and
5. add the lowest-cost legal candidate as a directed field RS arc, if one exists.

Thus

$$a_{ij}^{\mathrm{RS}} \in A$$

if and only if the corresponding RS maneuver is considered physically legal. An RS arc stores the actual RS geometry γ_a, not merely its scalar distance.

The rear axle is the RS reference point. Its complete tractor footprint must remain inside the field polygon after irrigation channels are removed, with clearance from tree trunks. The rear-axle trace must remain outside the interior of the convex hull of tree locations in that field. This approximation keeps RS maneuvers in headlands and sidelands rather than cutting between trees. A reversing field maneuver remains legal unless the forward-only field option is selected; road-junction turns are always forward-only.

8.1 Sampled RS turning radii

The tractor's turning radius $r_{\min}$ is a minimum feasible radius, so it bounds curvature from above rather than requiring the tractor to steer at that limit throughout a maneuver. The analytic RS solver generates paths for one specified radius at a time. The builder therefore samples

$$r_k = r_{\min} + k\Delta r, \qquad k = 0, \ldots, N,$$

with defaults $N = 2$ and $\Delta r = 0.35\,\mathrm{m}$. For the 2.0 m tractor trial this gives 2.0, 2.35, and 2.7 m. A path using the minimum radius may clip a channel or tree margin, while a wider turn at the same endpoint poses may clear it; forward-only road turns particularly benefit from this choice. Candidate paths from all sampled

radii are ranked by the same distance-equivalent movement cost and checked for swept-footprint feasibility. Only the lowest-cost legal path for an endpoint pair becomes a graph arc. Thus radius sampling broadens the finite candidate set, but does not solve a continuous-curvature optimal-control problem. The arc set can also change when RS scoring parameters change, even if a selected route has no reverse travel.

For a field not contained in the farm task's field list,

$$A_{\text{row}} = \emptyset, \qquad A_{\text{RS}} = \emptyset$$

for that field. Only its road network remains usable.

8.2 RS candidate pruning

The mathematically complete graph considers every relevant ordered endpoint pair in a selected field.

A practical implementation may use a configured maximum Euclidean separation

$$d_{\text{candidate}}(i, j) \leq R_{\text{RS}}$$

to avoid generating obviously undesirable long-range RS connections. Such pruning changes the feasible graph and therefore must be treated as an explicit planning policy rather than as an invisible optimization approximation.

9 Ordered Field-Service Stages

To enforce the task order $\mathcal{L} = (L_1, \ldots, L_m)$, construct a staged graph G^{ord}. For each stage i, copy the complete directed road graph, including its feasible junction turns. Add row-service and field RS arcs only for field L_i in that stage. Roads belonging to other fields remain available for transit.

A stage-switch arc is inserted only at a road state reached by an RS exit from field L_i:

$$(v, i) \longrightarrow (v, i + 1), \qquad i = 1, \ldots, m - 1.$$

It changes the routing state at the same tractor pose, has zero path length and zero cost, and can be used at most once. Switches run only forward through the stage sequence. StartPose lies in stage 1 and EndPose lies in stage m. For a single-field task, no copies or switches are required.

Because all service requirements must be met, and service arcs for L_i exist only in stage i, a valid route must finish that field's service before taking its switch. This preserves the requested field order while CP-SAT chooses road transit and field access points jointly. The switch is bookkeeping; it does not command a tractor motion.

10 Start and End

The graph must contain distinguished vertices

$$s \in V$$

and

$$t \in V$$

representing StartPose and EndPose.

The current implementation requires road-aligned start and end poses outside junction setbacks. Split the corresponding road geometry, preserve the pose heading, and connect each terminal to the road state in its permitted travel direction. If a pose cannot legally connect to the constructed graph, graph construction fails before CP-SAT is invoked. Field-interior terminal poses remain a possible future extension.

The special case

$$s = t$$

represents a closed mission.

11 Arc Costs

For each arc a,

$$\ell_a = \text{length}(\gamma_a).$$

Let r_a be reverse distance, g_a the number of internal forward/reverse gear changes, and e_a the number of contiguous reverse episodes within the stored maneuver. The nominal movement cost in distance-equivalent meters is

$$d_a = \ell_a + (\lambda_{\text{rev}} - 1)r_a + \lambda_{\text{gear}} g_a + \lambda_{\text{event}} e_a.$$

The current defaults are $\lambda_{\text{rev}} = 8$, $\lambda_{\text{gear}} = 2\,\text{m}$, and $\lambda_{\text{event}} = 100\,\text{m}$. Thus one meter of reverse travel costs eight times one meter of forward travel in the objective. The gear-change count is internal to an RS arc; the present model does not count a shift between two different arcs. Road-junction turns are forward-only and therefore have $r_a = g_a = e_a = 0$.

CP-SAT uses integer coefficients. With scale $S = 100$ cost units per meter, define

$$c_a = \max\{1, \text{round}(S d_a)\} \qquad \text{for movement arcs,}$$

and $c_a = 0$ for a zero-motion field-stage switch. Quantization and the positive-cost floor introduce only small differences from the nominal distance-equivalent score.

The model retains ℓ_a, r_a, g_a, and e_a for reporting and reconstruction while using c_a for CP-SAT. The same unrounded movement-cost terms rank feasible RS candidates for each endpoint pair before the graph is optimized. A separate repeat penalty, defined below, depends on the total number of uses of a physical crop lane and cannot be assigned to an individual arc independently.

Future versions may add costs for surfaces, field exits, energy use, or other operational considerations without changing the graph's basic service and flow constraints.

12 CP-SAT Decision Variables

For every directed arc $a \in A$, define an integer traversal-count variable

$$x_a \in \{0, 1, \ldots, M_a\}.$$

The interpretation is

$$x_a = \text{number of times the tractor traverses arc } a.$$

Road arcs, RS arcs, and row arcs may all be traversed more than once. A repeated row traversal is therefore represented naturally by

$$x_a > 1$$

or by use of both directional row arcs.

The model must provide a finite upper bound M_a because CP-SAT integer variables are bounded. For the initial implementation a conservative common bound may be used, for example

$$M_a = 2K + 2,$$

where

$$K = |\mathcal{Q}|.$$

This bound should be configurable, and the implementation should report if an optimal or feasible solution saturates an arc bound, since saturation may indicate that the bound is too restrictive.

13 Excess Crop-Lane Traversals

Let $\mathcal{H}$ be the set of physical crop lanes and let $A_h \subseteq A_{\text{row}}$ contain both directed row arcs for lane h, regardless of travel direction. Let $Q_h = \bigcup_{a \in A_h} C_a$. Define the number of passes required for its service as

$$b_h = \begin{cases} 0, & Q_h = \emptyset, \\ 1, & Q_h \subseteq C_a \text{ for some } a \in A_h, \\ 2, & \text{otherwise.} \end{cases}$$

For a two-sided implement, one pass normally covers both tree sides adjacent to a lane. For a one-sided implement, opposite directions can cover different sides and both may be required. The baseline b_h therefore excludes required service passes from the repeat count.

Introduce a nonnegative integer variable

$$y_h = \max\left\{0, \sum_{a \in A_h} x_a - b_h\right\} \qquad \forall h \in \mathcal{H}.$$

CP-SAT enforces this equality with a maximum-equality constraint. The repeat cost is $S\lambda_{\text{repeat}} \sum_h y_h$, where the current default is $\lambda_{\text{repeat}} = 50\,\text{m}$ per excess physical-lane traversal. This is a soft cost: an additional pass remains available when it saves enough other travel.

14 Service Constraints

For every service requirement $q \in \mathcal{Q}$, define

$$A(q) = \{a \in A : q \in C_a\}.$$

The required-service constraint is

$$\sum_{a \in A(q)} x_a \geq 1 \qquad \forall q \in \mathcal{Q}.$$

Thus every farming task must be serviced at least once. This formulation deliberately permits repeated service-path traversal. It also permits one physical traversal to satisfy several requirements when the same arc belongs to several $A(q)$ sets.

If

$$A(q) = \emptyset$$

for any required task, graph construction has produced an infeasible mission and CP-SAT should not be invoked.

15 Vehicle Flow Constraints

For a vertex v, define

$$\delta^+(v) = \{a \in A : u_a = v\},$$

and

$$\delta^-(v) = \{a \in A : v_a = v\}.$$

For an open route with $s \neq t$, define

$$b_v = \begin{cases} +1, & v = s, \\ -1, & v = t, \\ 0, & \text{otherwise.} \end{cases}$$

Impose

$$\sum_{a \in \delta^+(v)} x_a - \sum_{a \in \delta^-(v)} x_a = b_v \qquad \forall v \in V.$$

For a closed route, $s = t$, use

$$b_v = 0 \qquad \forall v.$$

These constraints provide the degree conditions required for the selected directed multigraph to possess an Euler trail from s to t, provided the selected graph is also connected.

16 Connectivity Constraints

Flow balance alone is insufficient because CP-SAT could otherwise construct multiple disconnected route components.

Introduce a binary variable

$$z_v \in \{0, 1\}$$

for every vertex, where

$$z_v = 1$$

means the vertex participates in the selected route.

For every incident arc a at vertex v, impose

$$x_a \leq M_a z_v.$$

Also impose

$$z_v \leq \sum_{a \in \delta^+(v) \cup \delta^-(v)} x_a.$$

Set

$$z_s = 1.$$

For a nontrivial open mission also set

$$z_t = 1.$$

Next introduce an artificial integer connectivity-flow variable

$$f_a \in \{0, \ldots, |V| - 1\}$$

for every arc. This flow has no physical interpretation. Its only purpose is to prove that every selected route vertex is reachable from the start vertex.

Artificial flow may use an arc only if the tractor uses that arc:

$$f_a \leq (|V| - 1) x_a \qquad \forall a \in A.$$

Every selected vertex other than the root consumes one unit of artificial flow:

$$\sum_{a \in \delta^-(v)} f_a - \sum_{a \in \delta^+(v)} f_a = z_v \qquad \forall v \neq s.$$

The start vertex supplies the total required artificial flow:

$$\sum_{a \in \delta^+(s)} f_a - \sum_{a \in \delta^-(s)} f_a = \sum_{v \in V \setminus \{s\}} z_v.$$

These constraints force all used vertices into one route component rooted at StartPose.

17 Objective

With $c_{\text{repeat}} = \text{round}(S\lambda_{\text{repeat}})$, the objective is

$$\min \left(\sum_{a \in A} c_a x_a + c_{\text{repeat}} \sum_{h \in \mathcal{H}} y_h \right).$$

The first term prices geometric travel and RS reversing behavior; the second prices crop-lane uses beyond required service. The default repeat penalty is 50 m per excess traversal. A forward-only route can still change when the solve time or graph candidate scoring changes: CP-SAT may return a different feasible incumbent before proving an optimum.

Unlike a TSP-style formulation, crop-row lengths are included in the objective.

This is necessary because a row may be traversed more than once, and different physical service paths may have different lengths.

18 Complete CP-SAT Model

The optimization problem is therefore

$$\begin{aligned}
\min \quad & \sum_{a \in A} c_a x_a + c_{\text{repeat}} \sum_{h \in \mathcal{H}} y_h \\
\text{subject to} \quad & \sum_{a \in A(q)} x_a \geq 1 && \forall q \in \mathcal{Q}, \\
& \sum_{a \in \delta^+(v)} x_a - \sum_{a \in \delta^-(v)} x_a = b_v && \forall v \in V, \\
& x_a \leq M_a z_{u_a} && \forall a \in A, \\
& x_a \leq M_a z_{v_a} && \forall a \in A, \\
& z_v \leq \sum_{a \in \delta^+(v) \cup \delta^-(v)} x_a && \forall v \in V, \\
& z_s = 1, \quad z_t = 1 \text{ if } s \neq t, \\
& f_a \leq (|V| - 1) x_a && \forall a \in A, \\
& \sum_{a \in \delta^-(v)} f_a - \sum_{a \in \delta^+(v)} f_a = z_v && \forall v \neq s, \\
& \sum_{a \in \delta^+(s)} f_a - \sum_{a \in \delta^-(s)} f_a = \sum_{v \neq s} z_v, \\
& y_h = \max\left\{ 0, \sum_{a \in A_h} x_a - b_h \right\}, && \forall h \in \mathcal{H}, \\
& x_a \in \{0, \ldots, M_a\}, && \forall a \in A, \\
& z_v \in \{0, 1\}, && \forall v \in V, \\
& f_a \in \{0, \ldots, |V| - 1\}, && \forall a \in A, \\
& y_h \in \left\{ 0, \ldots, \sum_{a \in A_h} M_a \right\}, && \forall h \in \mathcal{H}.
\end{aligned}$$

This model is applied to the staged directed graph, so field order and junction-turn feasibility are encoded by graph construction as well as by these constraints. For a single field, the stage construction reduces to the ordinary directed graph.

19 CP-SAT Construction Procedure

The implementation shall perform the following steps in order.

1. Parse and validate the FarmTask.
2. Generate the logical service requirement set $\mathcal{Q}$.
3. Generate candidate row-path geometry from tree-row geometry and ImplementRange.
4. Canonicalize geometrically duplicate physical row paths while retaining all logical service requirements.
5. Construct directed row traversal arcs and their service coverage sets C_a.
6. Construct the directed road graph with oriented setback states and feasible forward-only RS junction-turn arcs; omit the original position-only junction states.
7. Insert road/field interface points at row projections and configured offsets outside the junction setbacks.
8. Insert StartPose and EndPose into the appropriate graph geometry.
9. For each selected field, sample RS radii and generate penalized-cost-ranked maneuvers between candidate oriented interface states.
10. Reject field and road-turn candidates that violate swept-footprint, channel, tree, or tree-hull constraints.
11. Copy road states by field-service stage, retain only the current field's row and field-RS arcs in each stage, and add forward zero-motion stage switches.
12. Verify that every service requirement has at least one compatible service arc.
13. Assign integer movement costs c_a from length, reverse distance, gear changes, and reverse episodes.
14. Create CP-SAT traversal variables x_a.
15. Create crop-lane excess variables y_h and maximum-equality constraints.
16. Add service constraints.
17. Add start/end vehicle-flow constraints.
18. Add route-connectivity variables and constraints.
19. Add the movement and row-repeat objective.
20. Invoke CP-SAT.
21. If the solver status is neither FEASIBLE nor OPTIMAL, report route-generation failure.
22. Extract all nonzero traversal counts x_a.
23. Reconstruct an ordered Euler trail.
24. Convert the ordered arc sequence into the final geometric route.
25. Validate the resulting route independently of CP-SAT.

20 Approximate Python Model Construction

The implementation should correspond closely to the following structure.

```
from ortools.sat.python import cp_model

# GIS and RS geometry are built before the optimizer is entered.
base = build_routing_graph(farm, task, config)
graph = ordered_field_graph(base, task)
arcs = list(graph.arcs.values())
vertices = list(graph.vertices.values())
# Precompute limit, outgoing, incoming, row_lanes, and service_requirements.
S = config.cost_scale_per_m
model = cp_model.CpModel()

# Traversal counts
x = {
    a.id: model.new_int_var(0, limit[a.id], f"x_{a.id}")
    for a in arcs
}

# Service requirements
for q in service_requirements:
    compatible = [a for a in arcs if q in a.service_ids]
    model.add(sum(x[a.id] for a in compatible) >= 1)

# Vehicle flow
for v in vertices:
    outflow = sum(x[a.id] for a in outgoing[v.id])
    inflow = sum(x[a.id] for a in incoming[v.id])
    rhs = 0
    if start_id != end_id:
        rhs = 1 if v.id == start_id else -1 if v.id == end_id else 0
    model.add(outflow - inflow == rhs)

# Used-vertex variables
z = {
    v.id: model.new_bool_var(f"z_{v.id}")
    for v in vertices
}
for a in arcs:
    model.add(x[a.id] <= limit[a.id] * z[a.u])
    model.add(x[a.id] <= limit[a.id] * z[a.v])
for v in vertices:
    incident = incoming[v.id] + outgoing[v.id]
    model.add(z[v.id] <= sum(x[a.id] for a in incident))
model.add(z[start_id] == 1)
if end_id != start_id:
    model.add(z[end_id] == 1)

# Artificial connectivity flow
FMAX = len(vertices) - 1
f = {
    a.id: model.new_int_var(0, FMAX, f"f_{a.id}")
    for a in arcs
```

```
}
for a in arcs:
    model.add(f[a.id] <= FMAX * x[a.id])
for v in vertices:
    fin = sum(f[a.id] for a in incoming[v.id])
    fout = sum(f[a.id] for a in outgoing[v.id])
    if v.id != start_id:
        model.add(fin - fout == z[v.id])
root_demand = sum(z[v.id] for v in vertices if v.id != start_id)
root_in = sum(f[a.id] for a in incoming[start_id])
root_out = sum(f[a.id] for a in outgoing[start_id])
model.add(root_out - root_in == root_demand)

# Excess physical-lane traversals in either direction
excess = {}
for lane_id, lane_arcs in row_lanes.items():
    baseline = required_service_passes(lane_arcs)
    upper = sum(limit[a.id] for a in lane_arcs)
    excess[lane_id] = model.new_int_var(0, upper,
                                        f"row_excess_{lane_id}")
    model.add_max_equality(
        excess[lane_id],
        [sum(x[a.id] for a in lane_arcs) - baseline, 0]
    )

# Integer distance-equivalent objective
model.minimize(
    sum(arc_cost(a, config) * x[a.id] for a in arcs)
    + round(S * config.row_repeat_penalty_m) * sum(excess.values())
)

solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = solver_time_limit
status = solver.solve(model)
```

Production code should separate graph construction from CP-SAT construction. CP-SAT-specific code must not contain GIS or Reeds–Shepp geometry calculations.

21 Solver Output

CP-SAT returns traversal multiplicities

$$x_a^*$$

rather than an ordered route. It also returns a solver status, objective value, best lower bound, and elapsed solve time. `FEASIBLE` means a valid route was found without proof of optimality within the time limit; `OPTIMAL` proves the best route for the finite graph and its stated costs. Neither status asserts optimality over all possible continuous tractor trajectories. The gap between a feasible objective and the lower bound is reported relative to the found objective.

Construct a new directed multigraph

$$G^*$$

by inserting exactly x_a^* copies of every arc a for which

$$x_a^* > 0.$$

For example, if

$$x_a^* = 3,$$

the optimized route contains three distinguishable copies of that physical arc.

The flow and connectivity constraints guarantee the graph structure needed for an Euler trail from s to t.

22 Route Reconstruction

Use Hierholzer's algorithm, or an equivalent directed Euler-trail implementation, on G^* beginning at s.

The result is an ordered arc sequence

$$R = (a_1, a_2, \ldots, a_N)$$

such that

$$v_{a_i} = u_{a_{i+1}}$$

for every adjacent pair and every selected arc copy occurs exactly once.

For an open mission,

$$u_{a_1} = s, \qquad v_{a_N} = t.$$

For a closed mission, the Euler trail is an Euler circuit rooted at s.

The geometric tractor route is

$$\Gamma = \gamma_{a_1} \oplus \gamma_{a_2} \oplus \cdots \oplus \gamma_{a_N},$$

where $\oplus$ concatenates consecutive path geometries while removing duplicate endpoint samples.

23 Service Annotation of the Reconstructed Route

While traversing the ordered arc sequence, maintain a set

$$Q_{\text{done}}.$$

Whenever an arc a_i is encountered, add

$$C_{a_i}$$

to this set.

Thus,

$$Q_{\text{done}} \leftarrow Q_{\text{done}} \cup C_{a_i}.$$

At the end of reconstruction, verify

$$Q_{\text{done}} = \mathcal{Q}.$$

If an arc occurs more than once, the first compatible occurrence may be annotated as the servicing pass and later occurrences may be marked as deadhead traversals.

For the current inspection-task model, any compatible traversal is assumed to perform service automatically.

A future implementation may introduce explicit implement-on/off decision variables if traversal and service need to be distinguished operationally.

24 Route Output

The primary route output should retain arc semantics rather than immediately reducing the result to a single point array.

For example,

```
Route:
start_pose
end_pose
total_distance_m
optimality_status
best_objective_bound
reverse_distance_m
reverse_events
row_repeat_count
penalty_parameters
segments:
- arc_id
arc_type
field_id
geometry
distance_m
driving_directions
service_ids
occurrence_index
```

A derived continuous path may then be generated by concatenating segment geometry. This path can subsequently be resampled and converted to a ROS `nav_msgs/Path` or another representation required by the trajectory-following layer. The farm route optimizer determines the desired topological/geometric route. It does not replace the downstream vehicle trajectory controller.

25 Required Validation

After solving and route reconstruction, independently check:

1. The first route state agrees with StartPose.
2. The final route state agrees with EndPose.
3. Every consecutive pair of route segments is graph-connected.
4. Every service requirement is covered.
5. Every selected RS path lies within its allowed driving region.
6. Every road-junction turn is forward-only and its swept tractor footprint remains on permitted roads, crossings, headlands, or sidelands without entering an irrigation channel.
7. The tractor's swept footprint and rear-axle tree-hull exclusion are respected for field RS paths.
8. No row or field-transit arc appears in an unselected field.
9. Field service follows $L_1, \ldots, L_m$ in order; each stage switch joins identical road poses and has zero motion.
10. Arc occurrence counts in the reconstructed route equal the CP-SAT values x_a^*.
11. The reconstructed route distance equals
$$\sum_a \ell_a x_a^*$$
within numerical tolerance.

12. The integer CP-SAT objective agrees with

$$\sum_a c_a x_a^* + c_{\text{repeat}} \sum_h y_h^*.$$

13. Each y_h^* equals the positive excess of total lane traversals over b_h.

14. No route arc violates its directionality or service semantics.

26 Minimum Unit Tests

The implementation must include at least the following synthetic tests.

26.1 Single required row

One service row is connected to the start/end road network.

Verify that the row is serviced and that repeated traversal is permitted when required to return to the endpoint.

26.2 One-sided implement

The row is physically traversable in both directions, but only one direction satisfies the service requirement.

Verify that CP-SAT chooses the service-compatible direction at least once.

Verify that the opposite direction may still appear as deadhead travel.

26.3 Shared two-sided service path

Two logical service requirements are covered by one physical directed row arc.

Verify that one traversal satisfies both requirements.

26.4 Road bridge used repeatedly

Several required tasks are accessible only through one road segment.

Verify that CP-SAT may assign

$$x_a > 1$$

to that road arc.

26.5 Unselected-field shortcut

Create a geometrically shorter route through the interior of a field not present in `field_list`.

Because no field-interior arcs are created there, verify that the optimizer uses the road network instead.

26.6 Invalid RS connection

Create an RS candidate whose path leaves the allowed field polygon.

Verify that the arc is absent from the optimization graph.

26.7 Forward-only road junction

Construct a junction with oriented setback states and a crossing over an irrigation channel. Verify that every turn arc is forward-only, its swept footprint stays in the permitted road/crossing/field-margin region, and no position-only junction vertex bypasses the turn arcs. A crossing shorter than twice the setback uses a midpoint state.

26.8 Reverse maneuver scoring

Offer a shorter RS maneuver with reverse motion and a longer forward-only maneuver. Verify that reverse distance and reverse-episode penalties can select the forward path. Confirm that a road-junction turn never reverses.

26.9 Excess row traversal scoring

Offer a cheap repeat traversal of a physical lane and a longer road return. Verify that a sufficiently large repeat penalty selects the road. For a one-sided implement, confirm that opposite-direction passes needed to cover different tree sides have $y_h = 0$.

26.10 Ordered field service

Use a two-field task with road transit available in both stages. Verify that all service in the first field precedes the forward stage switch and all service in the second follows it, while the switch itself has zero movement.

26.11 Open mission

Use distinct StartPose and EndPose.
Verify that the reconstructed Euler trail begins at s and terminates at t.

26.12 Closed mission

Set StartPose equal to EndPose.
Verify that the reconstructed route is an Euler circuit.

27 Design Boundary

The route-planning subsystem consists of five explicit stages:

1. GIS/task geometry
2. physical directed routing graph and feasible RS arcs
3. ordered field-stage expansion
4. CP-SAT arc optimization
5. Euler route reconstruction

The geometric graph builder decides what movements are physically and semantically legal. CP-SAT decides how many times each legal movement is used. Euler reconstruction converts those traversal counts into a specific ordered route. A downstream trajectory-following subsystem is responsible for executing that route on the tractor.

These responsibilities must remain separate in the implementation.